شرح سطر بسطر ٣١ — نظام التصميم الجديد (الطابع، الحافة السفلية، الأزرار)

مشروع رِزمة · Flutter · كل سطر مع سبب كتابته بهذه الطريقة

التصميم الجديد («رِزمة — التصميم الجديد» في Figma) يقوم على ثلاث أفكار: ألوان أوضح للأطفال، و«حافة سفلية» (Ledge) تجعل الأزرار والبطاقات تبدو ملموسة، و«طابع» يتغيّر حسب عمر الطالب: مرح للصفوف 1–6 وهادئ للصفوف 7–12.

لم نغيّر كل شاشة على حدة؛ غيّرنا المكوّنات المشتركة (الألوان، الزر، البطاقة، الشريط العلوي) فانتقل الشكل الجديد إلى كل الشاشات التي تستعملها، ثم عدّلنا الشاشات الرئيسية لتطابق ترتيب التصميم. هذا الملف يشرح المكوّنات الجديدة سطرًا بسطر.

lib/core/theme/rizma_look.dart — الطابع حسب العمر

lib/core/theme/rizma_look.dart

الأسطر 1–3

```
1   import 'package:flutter/material.dart';
2
3   import 'app_colors.dart';
```

ماذا يفعل؟ استيراد Material (لـ `ThemeExtension`) وألوان التطبيق.

لماذا بهذه الطريقة؟ الطابع امتداد للثيم، فيحتاج أنواع Material.

الأسطر 5–9

```
5   /// «طابع» الواجهة حسب عمر الطالب، من قواعد النسخة المعتمدة في التصميم:
6   /// الصفوف 1–6 مرحة (4 سماوي، حافة سفلية غامقة 56px، أزرار (بلا حدود،
7   /// والصفوف 7–12 هادئة (2 كريمي، حافة خفيفة 1.5px، حد رفيع 52px، أزرار).
8   ///
9   /// يُقرأ من الثيم: `RizmaLook.of(context)`. يبقى مرحًا قبل اختيار ملف.
```

ماذا يفعل؟ تعليق يلخّص جدول «قواعد النسخة المعتمدة» في التصميم: الفرق بين الطابعين.

لماذا بهذه الطريقة؟ من يقرأ الكود يعرف القاعدة دون فتح Figma.

الأسطر 10–11

```
10  @immutable
11  class RizmaLook extends ThemeExtension<RizmaLook> {
```

ماذا يفعل؟ `@immutable` وصنف يرث `ThemeExtension<RizmaLook>`.

لماذا بهذه الطريقة؟ `ThemeExtension` هو الطريقة الرسمية في Flutter لإضافة قيم خاصة بنا إلى الثيم، فتصل لكل ودجت عبر `Theme.of(context)` بلا تمرير معاملات.

الأسطر 12–18

```
  const RizmaLook._({
    required this.calm,
    required this.background,
    required this.ledgeDepth,
    required this.cardBorderWidth,
    required this.buttonHeight,
  });
```

ماذا يفعل؟ منشئ خاص بخمس قيم: هادئ؟، الخلفية، سماكة الحافة، سماكة حدّ البطاقة، ارتفاع الزر.

لماذا بهذه الطريقة؟ المنشئ خاص (`_.`) حتى لا يوجد إلا طابعان معرّفان أدناه، فلا تُخترع قيم عشوائية في الشاشات.

الأسطر 20–27

```
  /// الطابع المرح (الصفوف 1–6، والافتراضي).
  static const playful = RizmaLook._(
    calm: false,
    background: AppColors.bg,
    ledgeDepth: 4,
    cardBorderWidth: 0,
    buttonHeight: 56,
  );
```

ماذا يفعل؟ الطابع المرح: خلفية سماوية، حافة 4، بلا حدود، أزرار 56.

لماذا بهذه الطريقة؟ القيم من جدول التصميم حرفيًا («حافة 4px بلون غامق»، «أصغر زر 56»). الأزرار الأكبر أسهل لأصابع الصغار.

الأسطر 29–36

```
  /// الطابع الهادئ (الصفوف 7–12).
  static const quiet = RizmaLook._(
    calm: true,
    background: AppColors.cream,
    ledgeDepth: 2,
    cardBorderWidth: 1.5,
    buttonHeight: 52,
  );
```

ماذا يفعل؟ الطابع الهادئ: خلفية كريمية، حافة 2، حدّ 1.5، أزرار 52.

لماذا بهذه الطريقة؟ الطلاب الأكبر يفضّلون واجهة أهدأ؛ الحدّ الرفيع يعوّض الحافة الخفيفة في فصل البطاقة عن الخلفية.

الأسطر 38–40

```
  /// الطابع المناسب لصف الطالب ([grade] فارغ = لا ملف بعد).
  static RizmaLook forGrade(int? grade) =>
      grade != null && grade > lastPlayfulGrade ? quiet : playful;
```

ماذا يفعل؟ `forGrade`: يعيد الهادئ إن كان الصف أكبر من 6، وإلا المرح (وكذلك حين لا يوجد ملف).

لماذا بهذه الطريقة؟ قبل اختيار ملف (الترحيب، الإنشاء) لا نعرف الصف، والأغلب أن المستخدم طفل، فالمرح هو الافتراضي الآمن.

الأسطر 42–43

```
42    /// أعلى صف في الطابع المرح.
43    static const lastPlayfulGrade = 6;
```

ماذا يفعل؟ ثابت آخر صف مرح = 6.

لماذا بهذه الطريقة؟ رقم القاعدة في مكان واحد يُقرأ في `forGrade`.

الأسطر 45–58

```
45    /// هادئ (7–12) أم مرح (1–6).
46    final bool calm;
47
48    /// لون خلفية الشاشات.
49    final Color background;
50
51    /// سماكة الحافة السفلية للأزرار والبطاقات.
52    final double ledgeDepth;
53
54    /// سماكة حدّ البطاقات البيضاء (0 = بلا حد).
55    final double cardBorderWidth;
56
57    /// ارتفاع الزر العريض.
58    final double buttonHeight;
```

ماذا يفعل؟ الحقول الخمسة مع تعليق لكل منها.

لماذا بهذه الطريقة؟ حقول `final` لأن الصنف غير قابل للتغيير (`@immutable`).

الأسطر 60–61

```
60    /// لون حافة البطاقة البيضاء: أغمق في المرح، أخف في الهادئ.
61    Color get cardLedge => calm ? AppColors.line : AppColors.sky2;
```

ماذا يفعل؟ `cardLedge`: لون حافة البطاقة البيضاء — `line` الخفيف في الهادئ و`sky2` الأغمق في المرح.

لماذا بهذه الطريقة؟ البطاقة البيضاء على خلفية فاتحة تحتاج حافة ملوّنة لتظهر؛ اللون يتبع الطابع.

الأسطر 63–64

```
63    static RizmaLook of(BuildContext context) =>
64        Theme.of(context).extension<RizmaLook>() ?? playful;
```

ماذا يفعل؟ `of(context)`: يقرأ الطابع من الثيم، ويرجع المرح إن لم يوجد.

لماذا بهذه الطريقة؟ الرجوع للمرح يمنع الانهيار في الاختبارات أو أي شجرة بلا ثيمنا.

الأسطر 66–71

```
66    @override
67    RizmaLook copyWith() => this;
68
69    @override
70    RizmaLook lerp(RizmaLook? other, double t) =>
71        other == null || t < 0.5 ? this : other;
```

ماذا يفعل؟ `copyWith` و`lerp` المطلوبان من `ThemeExtension`. `lerp` ينتقل للطابع الآخر في منتصف الحركة.

لماذا بهذه الطريقة؟ لا قيم وسطية بين طابعين (حافة 3؟)؛ عند تبديل الملف يتبدّل الطابع دفعة واحدة.

السطر 72

```
72  }
```

ماذا يفعل؟ نهاية الصنف.

لماذا بهذه الطريقة؟ —

lib/core/widgets/ledge.dart — الحافة السفلية

lib/core/widgets/ledge.dart

الأسطر 1–4

```
1   import 'package:flutter/material.dart';
2
3   import '../theme/app_colors.dart';
4   import '../theme/rizma_look.dart';
```

ماذا يفعل؟ الاستيرادات: Material والألوان والطابع.

لماذا بهذه الطريقة؟ الحافة تعتمد سماكتها ولون البطاقة البيضاء على الطابع.

الأسطر 6–8

```
6   /// لون «الحافة السفلية» (Ledge) المناسب لكل لون تعبئة في التصميم:
7   /// الأخضر فوق أخضر غامق، والعنبري فوق عنبري غامق… والأبيض فوق سماوي.
8   Color ledgeEdgeFor(Color fill, RizmaLook look) {
```

ماذا يفعل؟ دالة `ledgeEdgeFor`: تعطي لون الحافة المناسب للون التعبئة.

لماذا بهذه الطريقة؟ في نظام التصميم لكل لون «حافة» محدّدة (Ledge/Teal، Ledge/Amber…)؛ دالة واحدة تضمن أن كل الشاشات تستعمل الأزواج نفسها.

الأسطر 9–18

```
    if (fill == AppColors.card) return look.cardLedge;
    if (fill == AppColors.teal) return AppColors.tealDark;
    if (fill == AppColors.amber) return AppColors.amberDark;
    if (fill == AppColors.coral) return AppColors.coralDark;
    if (fill == AppColors.blue) return AppColors.blueDark;
    if (fill == AppColors.purple) return AppColors.purpleDark;
    if (fill == AppColors.tealTint) return AppColors.mint;
    if (fill == AppColors.amberTint) return AppColors.amberLine;
    if (fill == AppColors.coralTint) return const Color(0xFFF6C3AF);
    if (fill == AppColors.sky) return AppColors.sky2;
```

ماذا يفعل؟ الأزواج الثابتة: الأبيض ← حافة الطابع، الأخضر ← أخضر غامق، العنبري ← عنبري غامق، المرجاني والأزرق والبنفسجي ← درجاتها الغامقة، والألوان الفاتحة ← درجة أغمق قليلًا منها.

لماذا بهذه الطريقة؟ القيم من لوحة الألوان في التصميم؛ المقارنة بالمساواة تكفي لأن الألوان ثوابت.

الأسطر 19–20

```
    final hsl = HSLColor.fromColor(fill);
    return hsl.withLightness((hsl.lightness - 0.14).clamp(0.0, 1.0)).toColor();
```

ماذا يفعل؟ لأي لون آخر: تُنقص الإضاءة 14% في فضاء HSL.

لماذا بهذه الطريقة؟ احتياط عام لألوان المواد أو الألوان المستقبلية، بدل أن تظهر بلا حافة.

السطر 21

```
  }
```

ماذا يفعل؟ نهاية الدالة.

لماذا بهذه الطريقة؟ —

الأسطر 23–26

```
  /// صندوق بزوايا مدوّرة وحافة سفلية صلبة (ظل بلا تمويه) يعطي إحساس
  /// «الزر الملموس» في التصميم. يحجز مساحة الحافة أسفله فلا يتراكب مع ما تحته.
  ///
  /// [depth] (4 للمرح، 2 للهادئ). [edge] فارغ = حسب اللون. فارغ = سماكة الطابع.
```

ماذا يفعل؟ تعليق `LedgeBox` : صندوق بزوايا مدوّرة وحافة صلبة، يحجز مساحتها.

لماذا بهذه الطريقة؟ يشرح القرار الأهم: الحافة جزء من حجم الودجت، فلا تتراكب مع العنصر التالي.

الأسطر 27–38

```dart
class LedgeBox extends StatelessWidget {
  const LedgeBox({
    super.key,
    required this.child,
    this.color = AppColors.card,
    this.edge,
    this.radius = 20,
    this.depth,
    this.border,
    this.padding,
    this.clip = false,
  });
```

ماذا يفعل؟ الصنف ومعاملاته: المحتوى، لون التعبئة (أبيض افتراضيًا)، لون الحافة، نصف القطر، السماكة، الحدّ، الهامش الداخلي، والقصّ.

لماذا بهذه الطريقة؟ كل المعاملات اختيارية ما عدا المحتوى، فيكفي `LedgeBox(child: …)` لبطاقة بيضاء مطابقة للتصميم.

الأسطر 40–51

```dart
  final Widget child;
  final Color color;
  final Color? edge;
  final double radius;
  final double? depth;

  /// حدّ إضافي. فارغ = حدّ الطابع للبطاقات البيضاء (1.5 في الهادئ) أو بلا حد.
  final BoxBorder? border;
  final EdgeInsetsGeometry? padding;

  /// قصّ المحتوى على الزوايا (للصور والأشرطة الملاصقة للحافة).
  final bool clip;
```

ماذا يفعل؟ الحقول مع تعليقات للحدّ والقصّ.

لماذا بهذه الطريقة؟ القصّ اختياري لأنه مكلف قليلًا؛ نحتاجه فقط حين يلامس المحتوى الحواف (صور، `InkWell`).

الأسطر 53–57

```dart
  @override
  Widget build(BuildContext context) {
    final look = RizmaLook.of(context);
    final d = depth ?? look.ledgeDepth;
    final shape = BorderRadius.circular(radius);
```

ماذا يفعل؟ بداية البناء: قراءة الطابع، والسماكة (المُمرَّرة أو سماكة الطابع)، وشكل الزوايا.

لماذا بهذه الطريقة؟ السماكة الافتراضية من الطابع فتتغيّر كل البطاقات معًا بين 4 و2.

الأسطر 58–62

```
    final resolvedBorder =
        border ??
        (color == AppColors.card && look.cardBorderWidth > 0
            ? Border.all(color: AppColors.line, width: look.cardBorderWidth)
            : null);
```

ماذا يفعل؟ الحدّ النهائي: المُمرَّر، أو حدّ الطابع للبطاقة البيضاء في الهادئ، أو لا شيء.

لماذا بهذه الطريقة؟ قاعدة التصميم «حدود البطاقات: بدون حدود في المرح، خط رفيع 1.5 في الهادئ» تُطبَّق تلقائيًا.

الأسطر 63–66

```
    Widget inner = padding == null
        ? child
        : Padding(padding: padding!, child: child);
    if (clip) inner = ClipRRect(borderRadius: shape, child: inner);
```

ماذا يفعل؟ المحتوى الداخلي: مع هامش إن طُلب، ومقصوص على الزوايا إن طُلب.

لماذا بهذه الطريقة؟ نبني المحتوى خطوة خطوة بمتغيّر `inner` بدل شروط متداخلة.

الأسطر 68–75

```
    // الحدّ يُرسم فوق المحتوى (foreground) حتى لا يغطيه Material داخلي.
    if (resolvedBorder != null) {
      inner = DecoratedBox(
        position: DecorationPosition.foreground,
        decoration: BoxDecoration(borderRadius: shape, border: resolvedBorder),
        child: inner,
      );
    }
```

ماذا يفعل؟ إن وُجد حدّ نرسمه فوق المحتوى (`DecorationPosition.foreground`).

لماذا بهذه الطريقة؟ كثير من البطاقات فيها `Material` أبيض داخلي (للتموّج عند الضغط) يغطّي أي حدّ مرسوم خلفه؛ رسمه فوقه يبقيه ظاهرًا.

الأسطر 77–78

```
    return Padding(
      padding: EdgeInsets.only(bottom: d),
```

ماذا يفعل؟ هامش سفلي بسماكة الحافة.

لماذا بهذه الطريقة؟ الظل لا يأخذ مساحة في التخطيط؛ بدون هذا الهامش تتراكب الحافة مع البطاقة التالية.

الأسطر 79–91

```
      child: DecoratedBox(
        decoration: BoxDecoration(
          color: color,
          borderRadius: shape,
          boxShadow: d <= 0
              ? null
              : [
                  BoxShadow(
                    color: edge ?? ledgeEdgeFor(color, look),
                    offset: Offset(0, d),
                  ),
                ],
        ),
```

ماذا يفعل؟ صندوق التعبئة: اللون والزوايا، و `BoxShadow` بإزاحة رأسية بقدر السماكة وبلا تمويه.

لماذا بهذه الطريقة؟ ظل بلا تمويه يرسم «شريحة» صلبة تحت البطاقة بالضبط كما في التصميم، دون رسم شكل إضافي. سماكة صفر = بلا ظل.

الأسطر 92–96

```
        child: inner,
      ),
    );
  }
}
```

ماذا يفعل؟ المحتوى داخل الصندوق، ونهاية البناء والصنف.

لماذا بهذه الطريقة؟ —

lib/core/constants/learning_rules.dart — عتبة الإتقان الموحّدة

`lib/core/constants/learning_rules.dart`

الأسطر 1–3

```
/// قواعد التعلّم المشتركة بين الشاشات، من «الفكرة الكاملة — نظام الألعاب»
/// في التصميم. مكان واحد حتى لا تختلف العتبة من شاشة لأخرى.
library;
```

ماذا يفعل؟ تعليق المكتبة: قواعد التعلّم المشتركة من «نظام الألعاب» في التصميم، و `library;` .

لماذا بهذه الطريقة؟ كانت العتبة 60% في الرئيسية و80% في النتيجة وخطة المراجعة؛ ملف واحد يمنع هذا التناقض مستقبلًا.

الأسطر 5–6

```
/// «المفهوم تحت هذا الإتقان يدخل المراجعة، وعنده أو فوقه يُعدّ «متقنًا».
const double kMasteryThreshold = 0.6;
```

ماذا يفعل؟ `kMasteryThreshold = 0.6` : تحتها يدخل المفهوم المراجعة، وعندها أو فوقها يُعدّ متقنًا.

لماذا بهذه الطريقة؟ القيمة من التصميم («المفهوم تحت 60% يدخل المراجعة»). تستعملها الرئيسية (`kReviewMasteryThreshold`) والنتيجة (`masteredAt`) وخطة المراجعة (`ReviewPlanner.target`).

lib/core/widgets/offline_chip.dart — شارة «دون إنترنت»

lib/core/widgets/offline_chip.dart

الأسطر 1–6

```dart
import 'package:flutter/material.dart';

import '../strings/ar.dart';
import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import 'app_icon.dart';
```

ماذا يفعل؟ الاستيرادات: النص والألوان والأنماط والأيقونة.

لماذا بهذه الطريقة؟ —

الأسطر 8–11

```dart
/// شارة «دون إنترنت» الصغيرة. قاعدة التصميم: تظهر مرة واحدة في الشريط
/// العلوي، لا في كل بطاقة.
class OfflineChip extends StatelessWidget {
  const OfflineChip({super.key});
```

ماذا يفعل؟ الصنف وتعليق القاعدة: تظهر مرة واحدة في الشريط العلوي.

لماذا بهذه الطريقة؟ من «قواعد المحتوى» في التصميم: تكرار الحالة على كل بطاقة يزحم الشاشة بلا معلومة جديدة.

الأسطر 13–20

```dart
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsetsDirectional.fromSTEB(10, 3, 10, 3),
      decoration: BoxDecoration(
        color: AppColors.tealTint,
        borderRadius: BorderRadius.circular(999),
      ),
```

ماذا يفعل؟ كبسولة بخلفية خضراء فاتحة وهامش صغير.

لماذا بهذه الطريقة؟ زوايا 999 = كبسولة بأي عرض.

الأسطر 21–32

```
        child: Row(
          mainAxisSize: MainAxisSize.min,
          spacing: 4,
          children: [
            Flexible(
              child: Text(
                ArStrings.offlineChip,
                maxLines: 1,
                overflow: TextOverflow.ellipsis,
                style: AppTextStyles.caption12.copyWith(color: AppColors.teal),
              ),
            ),
```

ماذا يفعل؟ صف: النص (سطر واحد يُقصّ إن ضاق) داخل `Flexible`.

لماذا بهذه الطريقة؟ `Flexible` + قصّ النص يمنعان الفيض على الشاشات الضيقة أو بخط مكبّر.

الأسطر 33–38

```
            const AppIcon('cloudoff', size: 14, color: AppColors.teal),
          ],
        ),
      );
    }
  }
```

ماذا يفعل؟ أيقونة «سحابة مقطوعة» صغيرة بنفس اللون، ونهاية الصنف.

لماذا بهذه الطريقة؟ الأيقونة من مجموعة التصميم نفسها.

lib/core/widgets/rizma_button.dart — الزر العريض

`lib/core/widgets/rizma_button.dart`

الأسطر 1–8

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import '../theme/rizma_look.dart';
import 'app_icon.dart';
import 'ledge.dart';
import 'pressable_scale.dart';
```

ماذا يفعل؟ الاستيرادات، ومنها الطابع والحافة.

لماذا بهذه الطريقة؟ —

الأسطر 10–32

```dart
/// أنواع الأزرار العريضة (الأزرار في «نظام التصميم»).
enum ButtonKind {
  /// أخضر ممتلئ بنص أبيض (الإجراء الرئيسي، زر واحد بكل شاشة).
  filled,

  /// أبيض بنص أخضر (الإجراء الثانوي).
  outlined,

  /// خلفية خضراء فاتحة بنص أخضر.
  soft,

  /// خلفية سماوية بنص داكن.
  neutral,

  /// حبري داكن (تأكيد الإجابة في الاختبار).
  ink,

  /// عنبري ممتلئ (التلميح الأوضح، المهام المميّزة).
  amber,

  /// مرجاني ممتلئ (المراجعة).
  coral,
}
```

ماذا يفعل؟ أنواع الزر السبعة مع تعليق لكل نوع، ومنها الجديد `coral` (المراجعة)، وصار `amber` عنبريًا ممتلئًا بعد أن كان أبيض بحدّ عنبري.

لماذا بهذه الطريقة؟ أسماء الأدوار لا الألوان، فتبقى الشاشات صحيحة لو تغيّر اللون. العنبري الممتلئ هو «تلميح أوضح» في التصميم.

الأسطر 34–51

```dart
/// زر عريض بزوايا 16 وحافة سفلية ملموسة. ارتفاعه من الطابع (56 للصفوف
/// اسم أيقونة من مجموعة [icon]. [height] الصغيرة، 52 للكبيرة) ما لم يُحدَّد.
/// فارغ = معطّل. [onPressed] التصميم تظهر بعد النص (في جهة اليسار في RTL).
class RizmaButton extends StatelessWidget {
  const RizmaButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.kind = ButtonKind.filled,
    this.icon,
    this.height,
  });

  final String label;
  final VoidCallback? onPressed;
  final ButtonKind kind;
  final String? icon;
  final double? height;
```

ماذا يفعل؟ تعليق الصنف ومعاملاته: النص، الضغط (فارغ = معطّل)، النوع، الأيقونة، والارتفاع **الاختياري**.

لماذا بهذه الطريقة؟ الارتفاع صار اختياريًا ليأخذ قيمة الطابع (56 أو 52) تلقائيًا؛ الشاشات التي تحتاج ارتفاعًا خاصًا تمرّره.

الأسطر 53–56

```
53    @override
54    Widget build(BuildContext context) {
55      final look = RizmaLook.of(context);
56      final enabled = onPressed != null;
```

ماذا يفعل؟ بداية البناء: الطابع وحالة التفعيل.

لماذا بهذه الطريقة؟ —

الأسطر 57–65

```
57      final (bg, fg) = switch (kind) {
58        ButtonKind.filled => (AppColors.teal, AppColors.card),
59        ButtonKind.outlined => (AppColors.card, AppColors.teal),
60        ButtonKind.soft => (AppColors.tealTint, AppColors.tealDark),
61        ButtonKind.neutral => (AppColors.sky, AppColors.ink),
62        ButtonKind.ink => (AppColors.ink, AppColors.card),
63        ButtonKind.amber => (AppColors.amber, AppColors.amberInk),
64        ButtonKind.coral => (AppColors.coral, AppColors.card),
65      };
```

ماذا يفعل؟ لون الخلفية ولون النص لكل نوع (سجلّ من قيمتين).

لماذا بهذه الطريقة؟ تفكيك السجل `(bg, fg)` يجعل الجدول قصيرًا وواضحًا.

الأسطر 66–69

```
66      final edge = kind == ButtonKind.ink ? Colors.black : ledgeEdgeFor(bg, look);
67      final border = kind == ButtonKind.outlined
68          ? const BorderSide(color: AppColors.line, width: 1.5)
69          : BorderSide.none;
```

ماذا يفعل؟ لون الحافة (أسود للحبري، وإلا حسب اللون)، والحدّ الرفيع للزر الأبيض فقط.

لماذا بهذه الطريقة؟ الزر الأبيض على خلفية فاتحة يحتاج حدًّا خفيفًا ليُرى؛ البقية ممتلئة لا تحتاجه.

السطر 70

```
70      final depth = enabled ? look.ledgeDepth : 0.0;
```

ماذا يفعل؟ السماكة الفعلية: صفر حين يكون الزر معطّلًا.

لماذا بهذه الطريقة؟ الزر المعطّل مسطّح (بلا حافة) فيبدو «غير قابل للضغط».

الأسطر 72–76

```
72      return Semantics(
73        button: true,
74        enabled: enabled,
75        label: label,
76        excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : زر، حالته، ونصه.

لماذا بهذه الطريقة؟ قارئ الشاشة يعلن زرًا واحدًا بنصه بدل قراءة عناصره الداخلية.

الأسطر 77–79

```
        // نحجز مساحة الحافة دائمًا فلا يتغيّر ارتفاع الزر حين يُعطَّل.
        child: Padding(
          padding: EdgeInsets.only(bottom: look.ledgeDepth),
```

ماذا يفعل؟ هامش سفلي بسماكة الطابع **دائمًا**.

لماذا بهذه الطريقة؟ لو حُجزت المساحة فقط حين يكون مفعّلًا لقفزت الشاشة عند تفعيل الزر.

الأسطر 80–86

```
          child: DecoratedBox(
            decoration: BoxDecoration(
              borderRadius: BorderRadius.circular(16),
              boxShadow: depth == 0
                  ? null
                  : [BoxShadow(color: edge, offset: Offset(0, depth))],
            ),
```

ماذا يفعل؟ الظل الصلب تحت الزر (حين يكون مفعّلًا).

لماذا بهذه الطريقة؟ نفس تقنية `LedgeBox`، لكن هنا مكتوبة مباشرة لأن الزر يحتاج `Material` بشكل مدوّر وحدّ.

الأسطر 87–93

```
            child: Material(
              color: enabled ? bg : AppColors.disabled,
              shape: RoundedRectangleBorder(
                borderRadius: BorderRadius.circular(16),
                side: enabled ? border : BorderSide.none,
              ),
              clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ `Material` باللون (أو الرمادي المعطّل) وشكل مدوّر 16 وحدّ إن وُجد، مع قصّ.

لماذا بهذه الطريقة؟ `Material` ضروري ليظهر تموّج `InkWell` فوق اللون، والقصّ يحصر التموّج في الزوايا.

الأسطر 94–99

```
              child: InkWell(
                onTap: onPressed,
                child: SizedBox(
                  height: height ?? look.buttonHeight,
                  child: Padding(
                    padding: const EdgeInsets.symmetric(horizontal: 20),
```

ماذا يفعل؟ `InkWell` للضغط، وصندوق بارتفاع الزر (المُمرَّر أو قيمة الطابع)، وهامش أفقي 20.

لماذا بهذه الطريقة؟ —

الأسطر 100–113

```
                  child: Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    spacing: 8,
                    children: [
                      Flexible(
                        child: Text(
                          label,
                          maxLines: 1,
                          overflow: TextOverflow.ellipsis,
                          style: AppTextStyles.button16.copyWith(
                            color: enabled ? fg : AppColors.card,
                          ),
                        ),
                      ),
```

ماذا يفعل؟ صف في الوسط: النص بنمط الأزرار الجديد (ExtraBold/17)، سطر واحد يُقصّ إن ضاق.

لماذا بهذه الطريقة؟ نص الزر لا يلتفّ لسطرين حتى لا يتغيّر ارتفاعه.

الأسطر 114–120

```
                      if (icon != null)
                        AppIcon(
                          icon!,
                          size: 20,
                          color: enabled ? fg : AppColors.card,
                        ),
                    ],
```

ماذا يفعل؟ الأيقونة بعد النص إن وُجدت.

لماذا بهذه الطريقة؟ في RTL تظهر يسار النص كما في التصميم.

الأسطر 121–130

```
                  ),
                ),
              ),
            ),
          ),
        ),
      ).pressable(enabled: enabled),
    );
  }
}
```

ماذا يفعل؟ الإغلاق، و`.pressable` لتصغير الزر قليلًا عند الضغط، ونهاية الصنف.

لماذا بهذه الطريقة؟ إحساس «ضغطت شيئًا» للأطفال؛ معطّل حين يكون الزر معطّلًا.

lib/core/widgets/primary_button.dart — الزر الرئيسي

`lib/core/widgets/primary_button.dart`

الأسطر 1–8

```dart
import 'package:flutter/material.dart';
import 'package:flutter_svg/flutter_svg.dart';

import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import '../theme/rizma_look.dart';
import 'ledge.dart';
import 'pressable_scale.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ —

الأسطر 10–28

```dart
/// الزر الرئيسي العريض بحافة سفلية ملموسة. إذا كان [onPressed] فارغًا يظهر
/// معطّلًا. ارتفاعه من الطابع ما لم يُحدَّد [height].
class PrimaryButton extends StatelessWidget {
  const PrimaryButton({
    super.key,
    required this.label,
    required this.onPressed,
    this.iconAsset,
    this.height,
    this.backgroundColor = AppColors.teal,
    this.foregroundColor = AppColors.card,
  });

  final String label;
  final VoidCallback? onPressed;
  final String? iconAsset;
  final double? height;
  final Color backgroundColor;
  final Color foregroundColor;
```

ماذا يفعل؟ تعليق ومعاملات: النص، الضغط، أيقونة SVG، ارتفاع اختياري، ولونا الخلفية والنص.

لماذا بهذه الطريقة؟ يُستعمل حين نحتاج ألوانًا حرّة (مثل الزر الأبيض فوق البطاقة الخضراء)، بخلاف `RizmaButton` بأنواعه الثابتة.

الأسطر 30–39

```dart
  @override
  Widget build(BuildContext context) {
    final look = RizmaLook.of(context);
    final enabled = onPressed != null;

    return Semantics(
      button: true,
      enabled: enabled,
      label: label,
      excludeSemantics: true,
```

ماذا يفعل؟ البناء: الطابع والتفعيل و`Semantics`.

لماذا بهذه الطريقة؟ —

الأسطر 40–53

```
      child: Padding(
        padding: EdgeInsets.only(bottom: look.ledgeDepth),
        child: DecoratedBox(
          decoration: BoxDecoration(
            borderRadius: BorderRadius.circular(16),
            boxShadow: enabled
                ? [
                    BoxShadow(
                      color: ledgeEdgeFor(backgroundColor, look),
                      offset: Offset(0, look.ledgeDepth),
                    ),
                  ]
                : null,
          ),
```

ماذا يفعل؟ هامش الحافة، وظل صلب بلون `ledgeEdgeFor(الخلفية)` حين يكون مفعّلًا.

لماذا بهذه الطريقة؟ الزر الأبيض فوق الأخضر يأخذ حافة سماوية تلقائيًا من الدالة نفسها.

الأسطر 54–63

```
          child: Material(
            color: enabled ? backgroundColor : AppColors.disabled,
            borderRadius: BorderRadius.circular(16),
            clipBehavior: Clip.antiAlias,
            child: InkWell(
              onTap: onPressed,
              child: Container(
                height: height ?? look.buttonHeight,
                padding: const EdgeInsets.symmetric(horizontal: 20),
                alignment: Alignment.center,
```

ماذا يفعل؟ `Material` مدوّر و`InkWell`، وحاوية بارتفاع الزر ومحتوى في الوسط.

لماذا بهذه الطريقة؟ —

الأسطر 64–81

```
                child: Row(
                  mainAxisSize: MainAxisSize.min,
                  spacing: 8,
                  children: [
                    if (iconAsset != null)
                      SvgPicture.asset(iconAsset!, width: 20, height: 20),
                    Flexible(
                      child: Text(
                        label,
                        maxLines: 1,
                        overflow: TextOverflow.ellipsis,
                        style: AppTextStyles.button16.copyWith(
                          color: foregroundColor,
                        ),
                      ),
                    ),
                  ],
                ),
```

ماذا يفعل؟ صف: الأيقونة ثم النص (يُقصّ إن ضاق).

لماذا بهذه الطريقة؟ الأيقونة أولًا فتظهر يمين النص في RTL كما في التصميم الأصلي لهذا الزر.

الأسطر 82–89

```
              ),
            ),
          ),
        ),
      ),
    ).pressable(enabled: enabled);
  }
}
```

ماذا يفعل؟ الإغلاق و`.pressable` ونهاية الصنف.

لماذا بهذه الطريقة؟ —

lib/core/widgets/rizma_card.dart — البطاقة

lib/core/widgets/rizma_card.dart

الأسطر 1–4

```
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import 'ledge.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ —

الأسطر 6–20

```
/// بطاقة بيضاء بزوايا 20 وحافة سفلية (Ledge/Card في التصميم).
/// في الطابع الهادئ تأخذ حدًا رفيعًا وحافة أخف تلقائيًا.
class RizmaCard extends StatelessWidget {
  const RizmaCard({
    super.key,
    required this.child,
    this.padding = const EdgeInsets.all(16),
    this.color = AppColors.card,
  });

  final Widget child;
  final EdgeInsetsGeometry padding;

  /// لون التعبئة (الأبيض افتراضيًا، أو لون فاتح لبطاقة مميّزة).
  final Color color;
```

ماذا يفعل؟ الصنف ومعاملاته: المحتوى والهامش واللون (جديد).

لماذا بهذه الطريقة؟ إضافة اللون تسمح ببطاقة فاتحة مميّزة دون صنف جديد.

الأسطر 22–26

```
  @override
  Widget build(BuildContext context) {
    return LedgeBox(color: color, padding: padding, child: child);
  }
}
```

ماذا يفعل؟ البناء: `LedgeBox` بنفس اللون والهامش.

لماذا بهذه الطريقة؟ البطاقة صارت غلافًا رفيعًا حول `LedgeBox`، فيبقى اسمها المعروف في الشاشات وتأخذ الشكل الجديد تلقائيًا.

lib/features/home/widgets/home_top_bar.dart — شريط الرئيسية العلوي

`lib/features/home/widgets/home_top_bar.dart`

الأسطر 1–9

```
import 'package:flutter/material.dart';

import '../../../core/db/app_database.dart';
import '../../../core/strings/ar.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
import '../../../core/widgets/app_icon.dart';
import '../../../core/widgets/mascot.dart';
import '../../../core/widgets/offline_chip.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ —

الأسطر 11–24

```
/// شريط الرئيسية العلوي من التصميم: الشعار مع التميمة (يمين)، وشارة
/// «دون إنترنت». [onAvatar] يفتح «حسابي». [onAvatarLongPress] وزر الحساب (يسار)
/// لأدوات التطوير (debug فقط).
class HomeTopBar extends StatelessWidget {
  const HomeTopBar({
    super.key,
    required this.profile,
    required this.onAvatar,
    this.onAvatarLongPress,
  });

  final Profile profile;
  final VoidCallback onAvatar;
  final VoidCallback? onAvatarLongPress;
```

ماذا يفعل؟ تعليق ومعاملات: الملف النشط، فتح «حسابي»، والضغطة المطوّلة لأدوات التطوير.

لماذا بهذه الطريقة؟ انتقلت أدوات التطوير إلى زر الحساب بعد أن صار الأفاتار القديم في التحية تميمة كبيرة.

الأسطر 26–32

```
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsetsDirectional.fromSTEB(16, 12, 16, 0),
      child: Row(
        spacing: 8,
        children: [
```

ماذا يفعل؟ صف بهامش 16.

لماذا بهذه الطريقة؟ —

الأسطر 33–37

```
          Mascot(size: 34, color: profile.color),
          Text(
            ArStrings.appName,
            style: AppTextStyles.heading22.copyWith(color: AppColors.teal),
          ),
```

ماذا يفعل؟ التميمة الصغيرة بلون الطالب وكلمة «رزمة» بالأخضر.

لماذا بهذه الطريقة؟ الشعار في اليمين كما في التصميم.

الأسطر 38–39

```
          const Spacer(),
          const Flexible(child: OfflineChip()),
```

ماذا يفعل؟ فاصل مرن ثم شارة «دون إنترنت» (مرنة لتنكمش على الشاشات الضيقة).

لماذا بهذه الطريقة؟ هذا هو المكان الوحيد للشارة في الرئيسية.

الأسطر 40–58

```
            Semantics(
              button: true,
              label: ArStrings.navProfile,
              excludeSemantics: true,
              child: GestureDetector(
                onTap: onAvatar,
                onLongPress: onAvatarLongPress,
                child: Container(
                  width: 40,
                  height: 40,
                  decoration: const BoxDecoration(
                    color: AppColors.teal,
                    shape: BoxShape.circle,
                  ),
                  alignment: Alignment.center,
                  child: const AppIcon('user', size: 20, color: AppColors.card),
                ),
              ),
            ),
```

ماذا يفعل؟ زر الحساب: دائرة خضراء 40 بأيقونة مستخدم بيضاء، `Semantics` باسم «حسابي»، ونقرة وضغطة مطوّلة.

لماذا بهذه الطريقة؟ `GestureDetector` بدل `InkWell` لأنه يدعم الضغطة المطوّلة بلا تموّج، ولا نحتاج التموّج على دائرة صغيرة.

الأسطر 59–63

```
          ],
        ),
      );
    }
}
```

ماذا يفعل؟ الإغلاق ونهاية الصنف.

لماذا بهذه الطريقة؟ —

lib/features/home/widgets/reward_chips.dart — بلاطتا النجوم والأيام

`lib/features/home/widgets/reward_chips.dart`

الأسطر 1–7

```
import 'package:flutter/material.dart';

import '../../../core/strings/ar.dart';
import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
import '../../../core/widgets/app_icon.dart';
import '../../../core/widgets/ledge.dart';
```

ماذا يفعل؟ الاستيرادات.

لماذا بهذه الطريقة؟ —

الأسطر 9–16

```
/// بلاطتا النجوم والأيام المتتالية (عنبري ومرجاني كما في التصميم).
/// بلاطة الأيام تظهر فقط إن كانت السلسلة أكبر من صفر.
class RewardChips extends StatelessWidget {
  const RewardChips({super.key, required this.stars, required this.streak});

  final int stars;
  final int streak;
```

ماذا يفعل؟ الصنف ومعاملاه: النجوم والسلسلة.

لماذا بهذه الطريقة؟ الاسم القديم `RewardChips` بقي حتى لا تتغيّر الشاشات والاختبارات.

الأسطر 17–23

```
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsetsDirectional.fromSTEB(20, 0, 20, 0),
      child: Row(
        spacing: 10,
        children: [
```

ماذا يفعل؟ صف ببلاطتين متساويتين.

لماذا بهذه الطريقة؟ `Expanded` يقسم العرض بالتساوي؛ بلاطة واحدة تأخذ العرض كله.

الأسطر 24–32

```
          Expanded(
            child: _Tile(
              icon: 'starf',
              label: ArStrings.stars(stars),
              tint: AppColors.amberTint,
              solid: AppColors.amber,
              text: AppColors.amberInk,
            ),
          ),
```

ماذا يفعل؟ بلاطة النجوم: خلفية عنبرية فاتحة ومربع عنبري ونص بلون `amberInk`.

لماذا بهذه الطريقة؟ النجوم تظهر دائمًا بعد أول نشاط.

الأسطر 33–42

```
            if (streak > 0)
              Expanded(
                child: _Tile(
                  icon: 'flame',
                  label: ArStrings.streakDays(streak),
                  tint: AppColors.coralTint,
                  solid: AppColors.coral,
                  text: AppColors.coralDark,
                ),
              ),
```

ماذا يفعل؟ بلاطة الأيام (مرجانية) فقط إن كانت السلسلة أكبر من صفر.

لماذا بهذه الطريقة؟ «0 أيام متتالية» محبطة ولا تفيد.

الأسطر 43–47

```
          ],
        ),
      );
    }
  }
```

ماذا يفعل؟ إغلاق الصف والصنف.

لماذا بهذه الطريقة؟ —

الأسطر 49–62

```
class _Tile extends StatelessWidget {
  const _Tile({
    required this.icon,
    required this.label,
    required this.tint,
    required this.solid,
    required this.text,
  });

  final String icon;
  final String label;
  final Color tint;
  final Color solid;
  final Color text;
```

ماذا يفعل؟ البلاطة الخاصة ومعاملاتها: الأيقونة والنص والألوان الثلاثة.

لماذا بهذه الطريقة؟ ألوان ممرّرة فتخدم البلاطتين بكود واحد.

الأسطر 64–72

```
64    @override
65    Widget build(BuildContext context) {
66      return LedgeBox(
67        color: tint,
68        radius: 18,
69        padding: const EdgeInsetsDirectional.fromSTEB(12, 10, 12, 10),
70        child: Row(
71          spacing: 10,
72          children: [
```

ماذا يفعل؟ `LedgeBox` بالخلفية الفاتحة وزوايا 18، وصف بفاصل 10.

لماذا بهذه الطريقة؟ حافتها تُحسب تلقائيًا (عنبري فاتح ← `amberLine`).

الأسطر 73–82

```
73            Container(
74              width: 36,
75              height: 36,
76              decoration: BoxDecoration(
77                color: solid,
78                borderRadius: BorderRadius.circular(10),
79              ),
80              alignment: Alignment.center,
81              child: AppIcon(icon, size: 20, color: AppColors.card),
82            ),
```

ماذا يفعل؟ مربع 36 باللون الممتلئ وأيقونة بيضاء.

لماذا بهذه الطريقة؟ من التصميم: الأيقونة داخل مربع ملوّن.

الأسطر 83–94

```
83            Expanded(
84              child: Text(
85                label,
86                maxLines: 2,
87                style: AppTextStyles.body15.copyWith(color: text),
88              ),
89            ),
90          ],
91        ),
92      );
93    }
94  }
```

ماذا يفعل؟ النص حتى سطرين، والإغلاق.

لماذا بهذه الطريقة؟ سطران يكفيان «يومان متتاليان» على الشاشات الضيقة بلا فيض.

lib/core/widgets/tag_chip.dart — الشارة (كبسولة نص وأيقونة)

lib/core/widgets/tag_chip.dart

مكوّن جديد مع الجولة الثانية من التصميم: الشارات الصغيرة تتكرّر في كل شاشة تقريبًا («أهلًا بك!»، «مكتملة»، «4 دروس»، «محمّل»...).

الأسطر 1–5

```dart
import 'package:flutter/material.dart';

import '../theme/app_colors.dart';
import '../theme/app_text_styles.dart';
import 'app_icon.dart';
```

ماذا يفعل؟ استيراد Material والألوان والخطوط والأيقونة.

لماذا بهذه الطريقة؟ كل ما تحتاجه الكبسولة.

الأسطر 7–9

```dart
/// شارة صغيرة (كبسولة) بنص وأيقونة اختيارية، كما تتكرّر في التصميم الجديد:
/// «أهلًا بك!»، «الدرس النشط»، «مكتملة»، «4 دروس»…
/// [outline] يرسمها بيضاء بحدّ ملوّن بدل التعبئة.
```

ماذا يفعل؟ تعليق: ما الشارة، وأن `outline` يرسمها بيضاء بحدّ ملوّن.

لماذا بهذه الطريقة؟ يشرح متى نستعمل الشكل المحدّد (مثل «المنهاج الفلسطيني» في الترحيب).

الأسطر 10–24

```dart
class TagChip extends StatelessWidget {
  const TagChip({
    super.key,
    required this.label,
    this.icon,
    this.background = AppColors.tealTint,
    this.foreground = AppColors.teal,
    this.outline = false,
  });

  final String label;
  final String? icon;
  final Color background;
  final Color foreground;
  final bool outline;
```

ماذا يفعل؟ الباني والحقول: النص مطلوب؛ الأيقونة اختيارية؛ الخلفية أخضر فاتح والنص أخضر افتراضيًّا؛ `outline` لا.

لماذا بهذه الطريقة؟ الافتراضي هو الشارة الأكثر تكرارًا في التصميم، فأغلب الاستدعاءات تمرّر النص فقط.

الأسطر 25–33

```dart
  @override
  Widget build(BuildContext context) {
    return Container(
      padding: const EdgeInsetsDirectional.fromSTEB(10, 3, 10, 3),
      decoration: BoxDecoration(
        color: outline ? AppColors.card : background,
        borderRadius: BorderRadius.circular(999),
        border: outline ? Border.all(color: foreground, width: 1.2) : null,
```

ماذا يفعل؟ `build`: حاوية بحشوة 10/3 وزوايا كاملة، بيضاء بحدّ إن كانت `outline` وإلا بلونها.

لماذا بهذه الطريقة؟ زوايا 999 تعطي كبسولة كاملة مهما كان الطول.

الأسطر 34–52

```
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        spacing: 4,
        children: [
          Flexible(
            child: Text(
              label,
              maxLines: 1,
              overflow: TextOverflow.ellipsis,
              style: AppTextStyles.caption12.copyWith(color: foreground),
            ),
          ),
          if (icon != null) AppIcon(icon!, size: 14, color: foreground),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ صف بأقل عرض: النص بسطر واحد مع «...» إن طال (`Flexible`)، ثم الأيقونة 14 بنفس اللون.

لماذا بهذه الطريقة؟ `Flexible` يسمح للشارة أن تضيق داخل صف ضيّق بدل أن تتجاوز الحدود. الأيقونة بعد النص كما في التصميم.

lib/features/onboarding/widgets/step_progress_card.dart — بطاقة تقدّم خطوات إنشاء الملف

`lib/features/onboarding/widgets/step_progress_card.dart`

جديدة: تظهر أعلى الخطوتين 1 و2 كما في التصميم 02 و03.

الأسطر 1–7

```
import 'package:flutter/material.dart';

import '../../../core/theme/app_colors.dart';
import '../../../core/theme/app_text_styles.dart';
import '../../../core/widgets/ledge.dart';
import '../../../core/widgets/progress_bar.dart';
import '../../../core/widgets/tag_chip.dart';
```

ماذا يفعل؟ استيراد Material والألوان والخطوط والحافة والشريط والشارة.

لماذا بهذه الطريقة؟ البطاقة مركّبة من مكوّنات مشتركة.

الأسطر 9–21

```
/// بطاقة تقدّم خطوات إنشاء الملف (التصميم 02 و03): «الخطوة 2 من 3: …» وشارة N
/// النسبة وشريط التقدّم.
class StepProgressCard extends StatelessWidget {
  const StepProgressCard({
    super.key,
    required this.label,
    required this.value,
  });

  final String label;

  /// 0..1
  final double value;
```

ماذا يفعل؟ تعليق وصنف `StepProgressCard`: النص («الخطوة 1 من 2: …») والقيمة من 0 إلى 1.

لماذا بهذه الطريقة؟ مكوّن واحد للخطوتين بقيمتين مختلفتين (0.5 و1).

الأسطر 23–32

```
  @override
  Widget build(BuildContext context) {
    final percent = (value * 100).round();
    return LedgeBox(
      radius: 20,
      padding: const EdgeInsetsDirectional.fromSTEB(14, 12, 14, 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 10,
        children: [
```

ماذا يفعل؟ `build`: النسبة المئوية، و`LedgeBox` بزوايا 20 وعمود بمسافة 10.

لماذا بهذه الطريقة؟ بطاقة بيضاء بحافة مثل باقي البطاقات.

الأسطر 33–47

```
          Row(
            spacing: 8,
            children: [
              Expanded(
                child: Text(
                  label,
                  style: AppTextStyles.label14.copyWith(color: AppColors.ink),
                ),
              ),
              TagChip(
                label: '$percent%',
                icon: percent >= 100 ? 'checkc' : 'bolt',
              ),
            ],
          ),
```

ماذا يفعل؟ صف: النص بلون الحبر، وشارة بالنسبة ببرق (أو علامة صح عند 100%).

لماذا بهذه الطريقة؟ علامة الصح عند الاكتمال تقول «آخر خطوة» دون كلام.

الأسطر 48–58

```
            ProgressBar(
              value: value,
              trackColor: AppColors.sky2,
              fillColor: AppColors.teal,
              height: 10,
            ),
          ],
        ),
      );
    }
  }
```

ماذا يفعل؟ شريط أخضر على مسار سماوي، ثم الإغلاق.

لماذا بهذه الطريقة؟ الشريط يكرّر النسبة بصريًّا للأطفال الذين لا يقرؤون النسب.